Kathmandu University

Department of Computer Science and Engineering
Dhulikhel, Kavre

A Project Report

on

"DSA based Disaster Management System"

[Code No: COMP 202]

(For partial fulfillment of Year II / Semester I in Computer Engineering)

Submitted by

Sushant Raj Sharma (69)

Submitted to

Mr. Sagar Acharya
Department of Computer Science and Engineering

February 24, 2026

This project work on

“DSA based Disaster Management System”

is the bona fide work of
“Sushant Raj Sharma(69)”
who carried out the project work under my supervision.

Project Teacher

_____

Er. Sagar Acharya
Teaching Assistant
Department of Computer Science and Engineering

Date: February 24, 2026

Acknowledgement

I would like to express my sincere gratitude to the **Department of Computer Science and Engineering** for providing me with the opportunity and necessary resources to complete my minor project titled *"DSA-Based Disaster Management System."* This project enabled me to apply theoretical concepts of Data Structures and Algorithms to a practical, real-world problem.

I am especially thankful to my teacher, **Mr. Sagar Acharya**, for his valuable guidance, continuous support, and constructive feedback throughout the project. His direction greatly helped me improve my understanding of algorithm implementation and structured problem-solving.

I also appreciate all faculty members and staff for maintaining a supportive academic environment. Lastly, I thank my family and friends for their encouragement and motivation during the completion of this project.

This experience has enhanced my technical knowledge and strengthened my confidence in applying DSA concepts effectively.

Sushant Raj Sharma (69)

Abstract

A Disaster Management System is designed to manage disaster reports, emergency response operations, and resource allocation in an organized and efficient manner. Such systems help maintain structured records of disaster incidents, affected areas, victim information, and available rescue resources. This project focuses on the development of a web-based application that supports disaster monitoring and rescue operation management using Data Structures and Algorithms.

The frontend of the system is developed using **HTML, CSS, and JavaScript**, providing an interactive and user-friendly interface for data entry and visualization. The backend processing is handled through **server.py** and **C++ (main.cpp)**, where core functionalities are implemented using fundamental DSA concepts such as Graphs, Priority Queues, Queues, and Trees. The system prioritizes disaster cases based on severity levels, calculates the shortest rescue routes using graph algorithms, and manages resource allocation efficiently.

In the proposed system, authorized users can add disaster details, manage victim records, allocate emergency resources, and analyze rescue routes. All operational data is processed systematically to ensure organized record management and optimized decision-making.
This project demonstrates the practical implementation of Data Structures and Algorithms, backend integration, and web-based system design in solving real-world disaster management challenges.

Keywords: Disaster Management System, Data Structures and Algorithms, C++, HTML, CSS, JavaScript, Server Integration, Graph Algorithms.

Contents

Acknowledgement **i**

Abstract **ii**

Table of Contents **iv**

List of Figures **v**

List of Tables **vi**

1 Introduction **2**

1.1 Background 2

1.2 Objectives 3

1.3 Motivation and Significance 3

2 Literature Review **4**

2.1 Introduction 4

2.2 Previous Works 4

2.3 Comparison of previous work. 5

2.4 Current Work 5

2.5 Technologies Used in Previous Works 6

2.6 Drawbacks and Gaps in Existing Systems 6

2.7 Summary 6

3 Methodology **7**

3.1 System Architecture 7

3.2 GUI Overview 8

3.2.1 Disaster Report Section. 9

3.2.2 Add Victim Section 9

3.2.3 Add Resources Selection 10

3.2.4 Queries and Report Section 11

3.3 System Requirements 12
3.3.1 Software Requirements 12
3.3.2 Hardware Requirements 13
3.4 Implementation Details 13
3.4.1 Software Implementation 13
3.5 Tools and Technologies Used 13
3.6 Summary 14

4 Discussion and Achievements 15
4.1 Introduction…….. 15
4.2 System Results…….…..15
4.3 Output screenshot
4.3.1 Main Interface of Disaster management system…….15
4.3.2 Disaster Priority list output……16
4.3.3 Show Victim Record……..…16
4.3.4 Apache Server Running(XAMPP)……17
4.4 Discussion……..17
4.5 Achievement……..18

5 Conclusion and Recommendation 19
5.1 Limitations 19
5.2 Future Enhancements 20

References 21

List of Figures

3.1 System Architecture of Disaster Management System 8
3.2 Add Disaster Report Screen 9
3.3 Victim Management Screen. 9
3.4 Resource Management Screen 10
3.5 Queries and Reports Section. 11

4.3.1 Main Interface of Disaster Management system . . .. ……15
4.3.2 Disaster Priority List Output. . . . . …….16
4.3.3 Show Victim Record . . . . . ……16
4.3.4 Apache server Running(XAMPP) 17

List of Tables

2.1 Comparison of Previous Works (Compact Version) 5

Abbreviations

- DSA – Data Structures and Algorithms
- GUI – Graphical User Interface
- HTML – Hyper Text Markup Language
- CSS – Cascading Style Sheets
- JS – JavaScript
- API – Application Programming Interface
- BFS – Breadth-First Search
- DFS – Depth-First Search
- OOP – Object-Oriented Programming
- JSON – JavaScript Object Notation
- UI – User Interface

Chapter 1

Introduction

1.1 Background

Natural disasters such as floods, earthquakes, landslides, and fires pose significant challenges to communities and emergency response authorities. Effective disaster management requires timely decision-making, proper resource allocation, and efficient route planning to minimize damage and save lives. Traditional manual systems often lead to delays, poor coordination, and inefficient use of resources during emergency situations.

With the advancement of computer technology, software-based disaster management systems have become essential tools for organizing information and supporting decision-making processes. By utilizing Data Structures and Algorithms (DSA), it becomes possible to handle large volumes of data efficiently, prioritize emergency cases based on severity, and determine the shortest rescue routes using graph-based techniques.

This project is developed to demonstrate how fundamental DSA concepts such as graphs, priority queues, queues, and search algorithms can be applied to build a structured and efficient Disaster Management System. The integration of a web-based frontend with a C++ backend further enhances the practical implementation of algorithmic solutions in real-world scenarios.

1.2 Objectives

The main objective of this project is to design and develop a **DSA-Based Disaster Management System** that efficiently manages disaster-related information and optimizes emergency response operations using fundamental data structures and algorithms.

The specific objectives of this project are:

- To develop a web-based interface using HTML, CSS, and JavaScript for managing disaster reports and rescue operations.
- To implement core backend logic using C++ for applying Data Structures and Algorithms.
- To represent affected areas using graph data structures for efficient route mapping.
- To implement Dijkstra's Algorithm to determine the shortest rescue path.
- To use a Priority Queue to handle disaster cases based on severity levels.

1.3 Motivation and Significance

The increasing frequency of natural disasters highlights the need for efficient and well-organized disaster response systems. Delays in decision-making, poor resource allocation, and lack of proper route planning can lead to severe consequences. This motivated the development of a DSA-Based Disaster Management System that applies fundamental data structures and algorithms to improve emergency handling and rescue operations.

The significance of this project lies in its practical application of concepts such as graphs, priority queues, and search algorithms to solve real-world problems. By integrating a web-based frontend with a C++ backend, the system demonstrates how theoretical DSA knowledge can be transformed into an effective and structured solution for disaster management, while also enhancing problem-solving and technical skills.

Chapter 2

Literature Review

2.1 Introduction

Disaster management systems play a crucial role in reducing the impact of natural and human-made disasters. Modern systems use technology to improve emergency response, resource allocation, and data organization. Efficient management depends heavily on proper data handling and optimized decision-making processes. Data Structures and Algorithms (DSA) are essential for prioritizing emergencies and finding shortest rescue routes. This chapter reviews existing approaches and highlights the importance of algorithm-based solutions in disaster management systems.

2.2 Previous Works

- **A. Kumar (2019)** developed a C++ console-based disaster reporting system that managed incident records and resource allocation using basic data structures. The system was functional but lacked a graphical interface and real-time route optimization features (Kumar, 2019).
- **OpenStreetMap Community (2024)** provides geographic data that supports route mapping and location tracking in emergency systems. While useful for navigation, it requires integration with additional algorithms for shortest path calculation and resource prioritization (OpenStreetMap, 2024).
- **Python Software Foundation (2024)** offers server-side frameworks for web-based emergency management applications. These frameworks simplify backend development but do not inherently focus on the implementation of core Data Structures and Algorithms (Python Software Foundation, 2024).

2.3 Comparison of Previous Work:

Source	Platform	Features	Limitations
A. Kumar (2019) (Kumar, 2019)	C++ (Console)	Disaster reporting; basic record management; resource tracking	No GUI; limited user interaction; no route optimization
OpenStreetMap Community (2024) (OpenStreetMap, 2024)	Geographic Data Platform	Map data; location tracking; route mapping support	Requires integration with algorithms; no built-in prioritization system
Python Software Foundation (2024) (PSF, 2024)	Web Backend Framework	Server-side processing; web integration; database support	Focuses on framework, not core DSA implementation
T. H. Cormen et al. (2009) (Cormen et al., 2009)	Textbook	Graph algorithms; shortest path; priority queues	Theoretical only; no complete disaster management system
R. Sharma (2023) (Sharma, 2023)	Web Portal	Online disaster reporting; alerts; monitoring dashboard	Database-heavy; limited emphasis on algorithm optimization

Table 2.1: Comparison of Previous Works (Compact Version)

2.4 Current Work

The proposed system is a web-based Disaster Management System integrated with a C++ backend that:

- Provides an interactive user interface using HTML, CSS, and JavaScript for managing disaster reports and rescue operations.

- Utilizes C++ to implement core Data Structures and Algorithms such as graphs, priority queues, and shortest path algorithms for efficient emergency handling.

- Integrates server-side communication through *server.py* to enable structured data exchange between the frontend and backend.

- Focuses on algorithm efficiency, organized data management, and optimized rescue route planning.

- Is suitable for academic purposes, demonstrating practical application of DSA concepts in solving real-world disaster management problems.

2.5 Technologies Used in Current Works

Modern disaster management systems commonly use web technologies and databases to store and manage incident information. Mapping platforms and graph-based techniques are applied for route planning and location tracking. Backend frameworks handle server-side processing, while user interfaces improve interaction and data visualization. Console-based systems focus on basic logic implementation, whereas advanced web portals provide real-time monitoring but often require internet access and complex infrastructure.

2.6 Drawbacks and Gaps in Existing Systems

• Manual disaster management systems are time-consuming and prone to human errors.

• Many digital systems depend heavily on continuous internet connectivity.

• Some systems focus mainly on data storage rather than efficient algorithm-based decision-making.

• Lack of proper implementation of route optimization and priority-based emergency handling.

• Limited integration of core Data Structures and Algorithms for efficient resource management.

2.7 Summary

The review of existing systems shows that while several disaster management solutions are available, many lack efficient algorithm-based decision-making and optimized resource handling. Some systems focus mainly on data storage or monitoring without proper route optimization and priority management. The proposed system addresses these gaps by integrating core Data Structures and Algorithms with a web-based interface and backend processing. It provides an organized, efficient, and practical solution for managing disaster reports, prioritizing emergencies, and optimizing rescue operations.

Chapter 3

Methodology

3.1 System Overview

The Disaster Management System is a web-based application developed using **HTML, CSS, and JavaScript** for the frontend, with backend processing handled through **server.py and C++ (main.cpp)**. The system is designed to manage disaster reports, prioritize emergencies, optimize rescue routes, and allocate resources efficiently using Data Structures and Algorithms.
The major components of the system include:
• **User Module:** Allows authorized users to report disasters, view affected areas, check priority levels, and analyze rescue routes.
• **Admin Module:** Enables management of disaster records, monitoring of emergency severity, and control over resource allocation.
• **Algorithm Module (C++ Backend):** Implements core DSA concepts such as Graphs, Dijkstra's Algorithm, Priority Queues, and Queues for route optimization and emergency handling.
• **Server Module:** Handles communication between the frontend and backend, ensuring structured data exchange and processing.

System Architecture:
The system follows a modular architecture where the frontend handles user interaction, the server manages communication, and the C++ backend performs algorithm-based processing. Each module operates independently while working together to ensure efficient disaster response management.

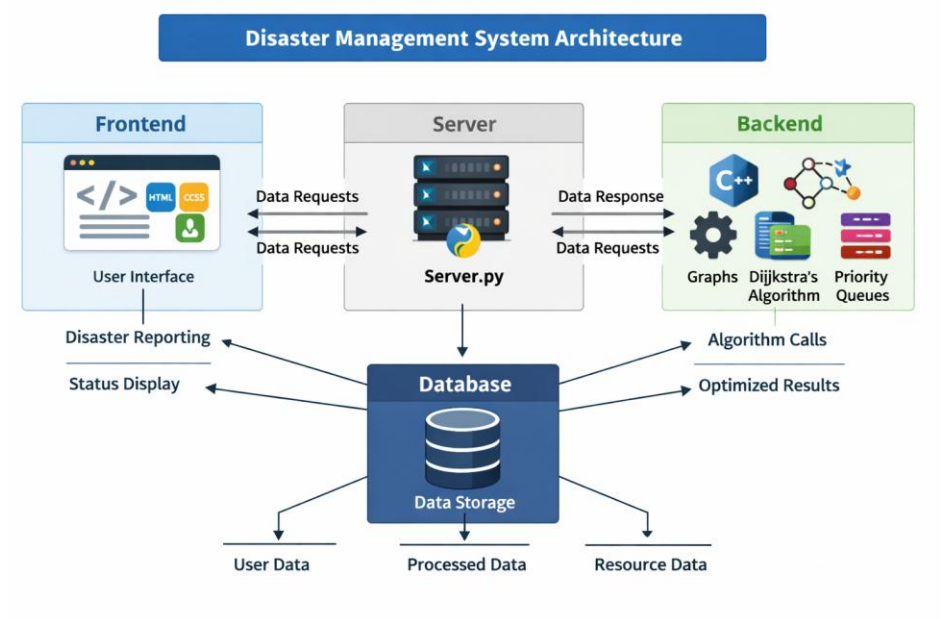

Figure 3.1: **Disaster management system architecture diagram.**

3.2 GUI Overview

The Graphical User Interface (GUI) of the system is designed to be simple, interactive, and user-friendly. It is developed using **HTML, CSS, and JavaScript**, providing a structured layout for managing disaster reports, victim records, and resource allocation.

The interface allows users to add disaster details, assign severity levels, manage emergency resources, and view algorithm-based outputs such as priority lists and shortest rescue paths. Clear buttons, input forms, and result display sections ensure smooth interaction between the user and the backend system.

The GUI focuses on ease of use while effectively supporting the core DSA-based functionalities of the application.

The interface is divided into multiple functional sections:

3.2.1 Add Disaster Report Section:

Allows users to enter disaster type, location, and severity level.(High severity level = more seriousness)

Figure 3.2: Disaster Report Section

3.2.2 Add Victim Section:

Enables entry of victim details such as name, age, location, and status.

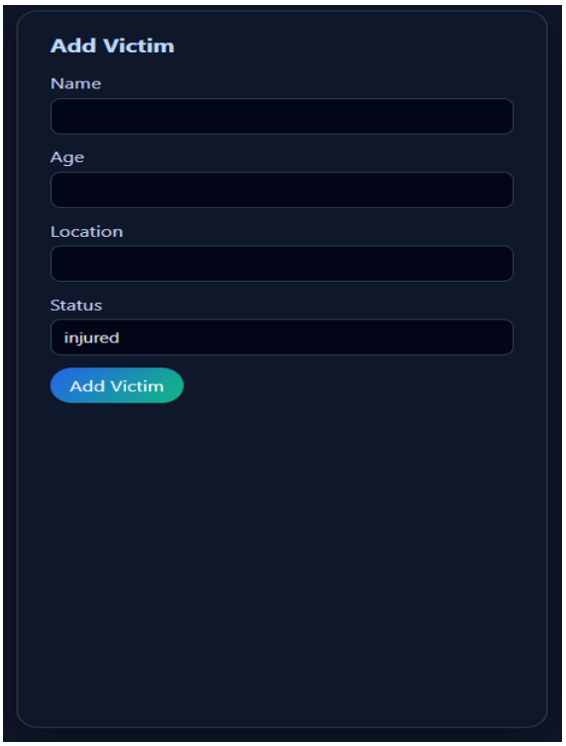

Figure 3.3: Add Victim Section

3.2.3 Add Resource Section:

Used to register available emergency resources such as ambulances and medical supplies.

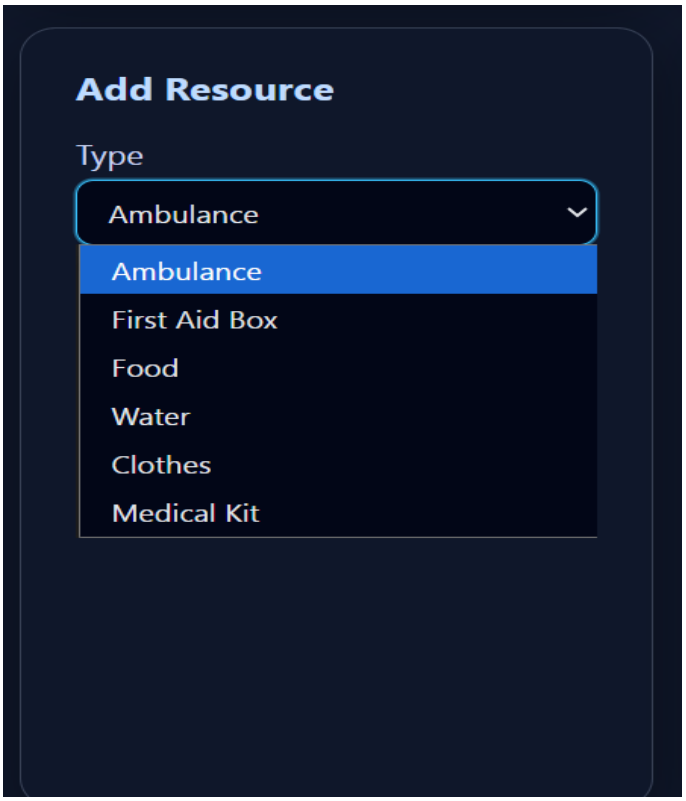

Figure 3.4: Add Resource Section

3.2.4 Queries & Reports Section:

Provides algorithm-based outputs including:

- Disaster Priority List
- Resource Display
- Victim List
- Greedy Resource Allocation
- Shortest Rescue Path (Dijkstra's Algorithm)
- BFS and DFS Traversal Demonstrations

Figure 3.4: Queries & Reports Section

3.3 System Requirements

3.3.1 Software Requirements

3.3.1.1 Programming Language

C++ (for backend algorithm implementation)

3.3.1.2 Frontend Technologies

HTML, CSS, and JavaScript (for user interface development)

3.3.1.3 Server-Side Technology

Python (server.py) with Apache Server (XAMPP) for backend communication

3.3.1.4 Operating System

Windows 10 or above

3.3.1.5 Development Tools

Visual Studio Code / Any C++ Compiler (e.g., MinGW or GCC)

3.3.2 Hardware Requirements

- 4 GB RAM
- 1 GHz Processor
- 500 MB free disk space
- 8 GB RAM
- 2 GHz or higher Processor
- 1 GB free disk space

3.4 Implementation Details

3.4.1 Software Implementation

The system was implemented using a web-based architecture with a C++ backend focused on Data Structures and Algorithms. The key implementation steps include:

3.4.1.2 Developing the Frontend Interface

The user interface was created using **HTML, CSS, and JavaScript**. Forms and interactive buttons were designed to allow disaster reporting, victim management, resource allocation, and algorithm-based queries.

3.4.1.3 Implementing Backend Logic in C++

Core functionalities such as disaster prioritization, shortest path calculation (Dijkstra's Algorithm), BFS/DFS traversal, and resource allocation were implemented in **C++** using appropriate data structures like graphs, priority queues, and queues.

3.4.1.4Server Integration

A **Python-based server (server.py)** was used to handle communication between the frontend and backend. Apache (XAMPP) was configured to enable structured data exchange and execution of backend processes.

3.5 Tools and Technologies Used

• **HTML, CSS, JavaScript:** For frontend interface development

• **C++:** For implementing core Data Structures and Algorithms

• **Python (server.py):** For server-side communication

• **Apache Server (XAMPP):** For hosting and backend execution

• **IDE/Compiler:** Visual Studio Code and GCC/MinGW Compiler

3.6 Summary

This chapter explained the methodology followed in developing the Disaster Management System. The system architecture, web-based GUI design, major modules, and backend algorithm implementation were discussed. The integration of HTML, CSS, JavaScript, Apache server, and C++ for DSA-based processing was also described. Figures, tables, and architectural diagrams provide a clear understanding of how the system operates in practice and how algorithmic concepts are applied to real-world disaster management scenarios.

Chapter 4

Discussion and Achievements

4.1 Introduction

This chapter presents the results obtained from the implementation of the DSA-Based Disaster Management System. It explains how the system performs disaster prioritization, resource allocation, and rescue route optimization using Data Structures and Algorithms. The functionality of each module is demonstrated through system outputs and interface screenshots.

4.2 System Results

The implemented system successfully performs the following operations:

- Disaster reports are added with severity levels.
- Emergencies are prioritized using a Priority Queue (Max Heap).
- Shortest rescue paths are calculated using Dijkstra's Algorithm.
- BFS and DFS traversals are executed for area connectivity analysis.
- Resources are allocated using queue-based or greedy approaches.
- Victim records are stored and displayed systematically.

The system correctly processes input data and returns optimized results through backend algorithm execution.

4.3 Output Screenshots:

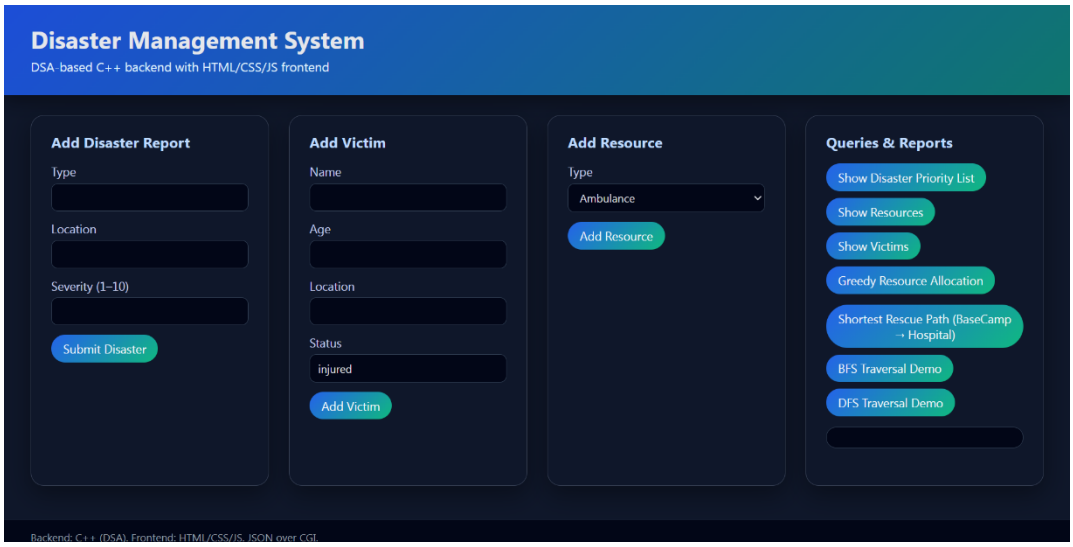

Figure 4.3.1: Main Interface of the Disaster Management System

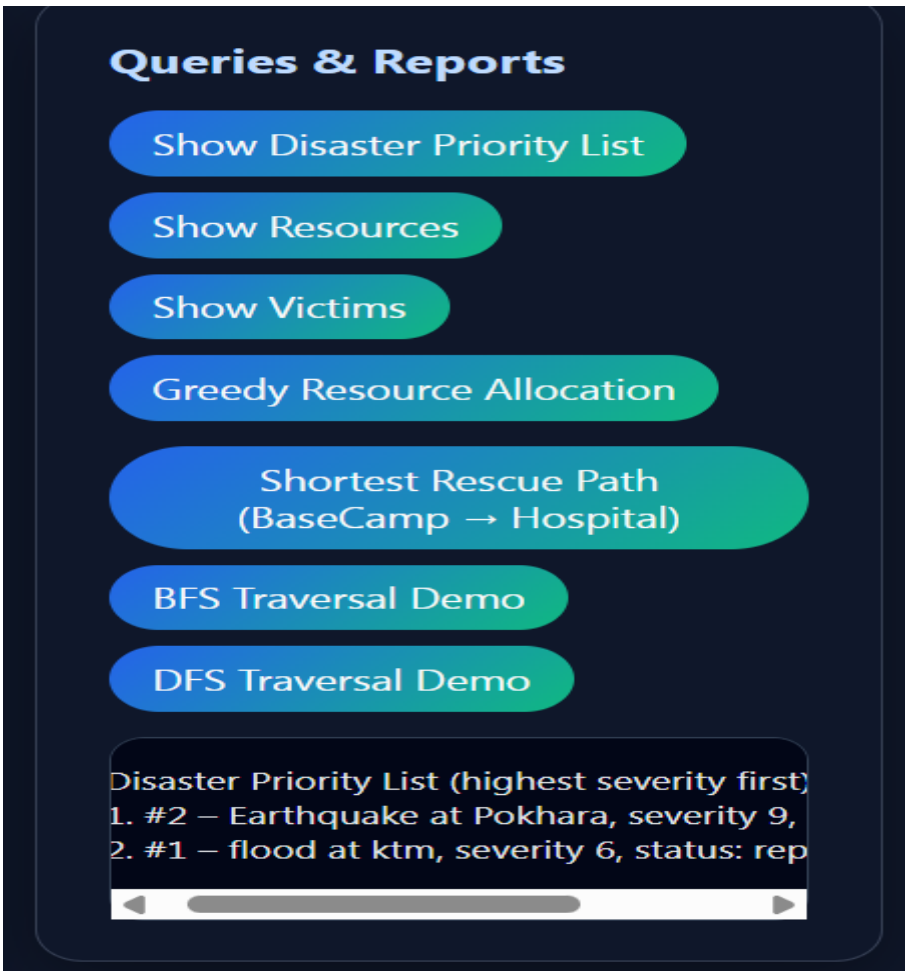

Figure 4.3.2: Disaster Priority List Output

Figure 4.3.3: Show Victim Record

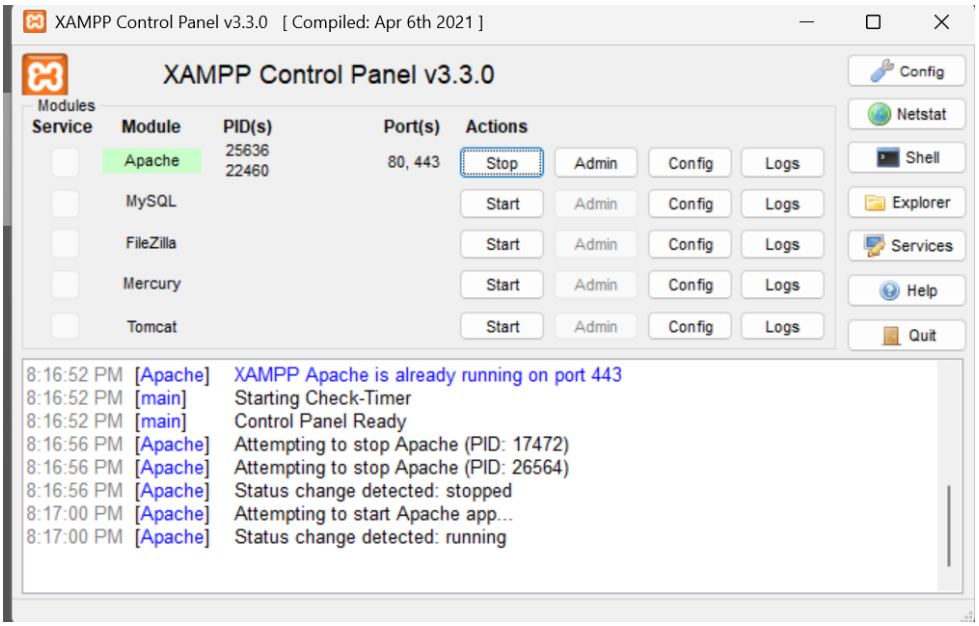

Figure 4.4.4: Apache Server Running (XAMPP)

4.4 Discussion

The results show that implementing core Data Structures and Algorithms significantly improves efficiency in disaster management tasks. Priority-based scheduling ensures high-severity cases are handled first. Graph algorithms provide optimized rescue routes, reducing response time. The modular integration of frontend, server, and backend ensures smooth system functionality.

The system demonstrates that theoretical DSA concepts can be effectively applied to real-world emergency management problems.

Achievements:

• Successfully developed a web-based Disaster Management System integrating frontend and backend technologies.
• Implemented core Data Structures and Algorithms such as Graphs, Dijkstra's Algorithm, Priority Queue, BFS, and DFS.
• Designed an interactive and user-friendly interface using HTML, CSS, and JavaScript.
• Integrated Apache server and backend processing for smooth communication between frontend and C++ logic.
• Achieved efficient disaster prioritization and optimized rescue route planning.
• Demonstrated practical application of theoretical DSA concepts in solving real-world problems.
• Improved problem-solving, backend development, and system integration skills through project implementation.

Chapter 5

Conclusion and Recommendation

This project successfully developed a web-based **Disaster Management System** that integrates a frontend interface with a C++ backend implementing core Data Structures and Algorithms. The main objectives outlined in Section 1.2 have been achieved, including disaster reporting, emergency prioritization, resource allocation, and shortest rescue route calculation.

The system provides a structured and user-friendly interface for managing disaster-related information. The implementation of graph algorithms such as Dijkstra's Algorithm ensures optimized rescue route planning, while the use of priority queues enables effective handling of high-severity emergencies. The integration of Apache server and backend processing ensures smooth communication between modules.

This project provided practical experience in algorithm implementation, backend integration, and system design. It demonstrates how theoretical DSA concepts can be applied to solve real-world disaster management challenges effectively.

5.1 Limitations

Although the system meets the primary objectives, some limitations exist:

- The system is designed for simulation purposes and does not include real-time GPS or live disaster data integration.
- It depends on a local server setup and does not support large-scale distributed deployment.
- Advanced authentication and role-based access control are limited.
- Real-time notifications and alert systems are not fully implemented.
- Concurrent multi-user scalability is limited in the current version.

5.2 Future Enhancements

The project can be further improved with the following enhancements:

• Integration of real-time GPS and live disaster monitoring systems.

• Deployment on a cloud-based server for large-scale multi-user access.

• Implementation of advanced security features such as multi-factor authentication.

• Addition of automated alert and notification systems (SMS/Email).

• Implementation of data analytics and visualization dashboards for better decision-making.

• Optimization of algorithms for handling larger datasets and real-time performance improvements.

Overall, the project demonstrates a structured, efficient, and scalable Disaster Management System and provides a strong foundation for future development and real-world deployment.

References:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.
- OpenStreetMap Contributors. (2024). *OpenStreetMap documentation*. Retrieved from https://www.openstreetmap.org
- Python Software Foundation. (2024). *Python documentation*. Retrieved from https://docs.python.org
- Apache Software Foundation. (2024). *Apache HTTP server documentation*. Retrieved from https://httpd.apache.org/docs
- Mozilla Developer Network (MDN). (2024). *HTML, CSS, and JavaScript documentation*. Retrieved from https://developer.mozilla.org
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.